



Global Jersey Store

E-Commerce Web Application

Technical Project Report

A full-featured international football jersey e-commerce platform

Author

Nossaiba El Asri

Date

July 2026

Portfolio Project

Table of Contents

- 1. Project Overview
- 2. Technologies Used
- 3. Architecture
 - 4.1 Product Catalog
 - 4.2 Product Pages
 - 4.3 Shopping Cart
 - 4.4 Checkout System
 - 4.5 Order Management (My Account)
 - 4.6 Authentication
 - 4.7 Wishlist
 - 4.8 UI/UX Design
 - 4.9 Admin Dashboard
- 5. Data Storage Explanation
- 6. Deployment & DevOps
- 7. Challenges & Solutions
- 8. Skills Demonstrated
- 9. Future Improvements
- 10. Conclusion

1. Project Overview

Global Jersey Store is a full-featured e-commerce web application built as a portfolio project to demonstrate full-stack web development skills using modern, industry-standard technologies. The platform allows users to browse, search, and purchase international football jerseys from national teams around the world, featuring a complete shopping experience from product discovery through checkout and order management.

Purpose	Portfolio project demonstrating full-stack web development skills — from responsive UI design to authentication, state management, and CI/CD deployment workflows.
Target Audience	Football fans worldwide and jersey collectors seeking national team jerseys in a clean, modern online store.
Project Type	E-commerce web application (portfolio / demo) — fully functional front-end with simulated checkout and order processing.
Live URL	https://global-jersey-store.vercel.app
GitHub	https://github.com/nossaiba-elas/global-jersey-store

2. Technologies Used

The project is built with a carefully selected stack of modern web technologies. Each technology was chosen for its industry relevance, developer experience, and suitability for a portfolio-grade e-commerce application. Below is a summary of every technology used and its role in the project.

Frontend Framework	Next.js 15 — React-based framework with the App Router, enabling file-based routing, server/client components, and optimized performance out of the box.
Language	TypeScript — strongly typed JavaScript that reduces runtime errors, improves developer productivity through autocomplete, and makes refactoring safer.
Styling	Tailwind CSS — utility-first CSS framework enabling rapid, consistent, and maintainable responsive styling without writing custom CSS files.
UI Components	@base-ui/react + shadcn/ui conventions — accessible, composable component primitives following the popular shadcn/ui design system patterns.
Icons	Lucide React — a clean, consistent, open-source icon set providing SVG icons throughout the UI.
State Management	Zustand with localStorage persistence — a lightweight, hook-based store for cart, wishlist, and orders, persisted across browser sessions via middleware.
Authentication	Firebase Authentication — Google OAuth 2.0 sign-in providing secure, managed user identity without building a custom auth backend.
Form Validation	Zod + React Hook Form — schema-based validation integrated with performant form state management for the checkout shipping form.
Deployment	Vercel — the platform behind Next.js, providing automatic CI/CD from GitHub, edge network distribution, and zero-downtime deployments.
Version Control	Git + GitHub — distributed version control with a remote repository as the single source of truth for all project code.

CI/CD Pipeline

GitHub Actions — automated workflows running ESLint linting and Next.js build checks on every push, ensuring code quality before deployment.

3. Architecture

3.1 Overview

Global Jersey Store follows a modern front-end-centric architecture built on the Next.js App Router. The application is purely client-side with no custom backend server — Firebase is used solely for authentication, and all product, cart, wishlist, and order data is managed either as static TypeScript constants or in browser-side state persisted to `localStorage`. This architectural decision keeps the stack intentionally simple, making the project easy to deploy and maintain as a portfolio demo while still showcasing real-world e-commerce functionality.

3.2 Architecture at a Glance

- **Next.js App Router** — file-based routing where each folder under `app/` maps to a URL segment. Pages, layouts, and nested routes are defined by convention (`page.tsx`, `layout.tsx`).
- **Client-side state with Zustand** — a single store (or small set of stores) manages cart, wishlist, and orders using React hooks, with Zustand's *persist* middleware automatically syncing to `localStorage`.
- **Static product data** — all jersey products and team metadata are defined as typed TypeScript constants in `products.ts` and `teams.ts`, imported directly into components.
- **Firebase for authentication only** — Firebase Authentication handles Google OAuth sign-in. No Firestore or Realtime Database is used; Firebase is not a data store in this project.
- **No backend server** — the application is purely a frontend deployed as static/server-rendered pages on Vercel. There is no Express, NestJS, or custom API layer.

3.3 Why This Architecture?

For a portfolio demo, the priority is to demonstrate the widest range of modern development skills — responsive UI, state management, authentication, routing, form validation, CI/CD — without the operational complexity of a full backend with a managed database. By using static product data and `localStorage`-backed state, the project avoids the need to provision, configure, and maintain a separate database server and API, keeping deployment as simple as a git push. This lets the focus remain on building a polished, feature-rich frontend that works reliably for visitors and reviewers. In a production setting, the architecture would evolve to include a real database and backend API (see Section 9: Future Improvements).

3.4 High-Level Data Flow

Browser (Client)	→	Next.js App Router (Pages & Layouts)
	→	Zustand Store (Cart / Wishlist / Orders)
	→	localStorage (Persistence via Zustand persist)
	→	Firebase Authentication (Google OAuth)
	→	Static TS Constants (products.ts / teams.ts)

4. Key Features

Global Jersey Store implements a comprehensive set of e-commerce features that together form a complete shopping experience. The following subsections detail each feature area, the functionality it provides, and the technical approach behind it.

4.1 Product Catalog

- 50+ international football jerseys from national teams across multiple continents and confederations.
- Filtering by team, league, price range, and size — allowing users to quickly narrow down to jerseys matching their preferences.
- Search functionality enabling users to find jerseys by name, team, or keyword.
- Sorting options including price (low to high / high to low), name (alphabetical), and trending (popularity-based ordering).

4.2 Product Pages

- Detailed individual product view displaying high-quality jersey imagery, team information, and pricing.
- Interactive size selection with visual size buttons and availability indicators.
- Related products section showing jerseys from the same team or category to encourage browsing.
- Add to cart and add to wishlist actions available directly from the product page.

4.3 Shopping Cart

- Add and remove items dynamically, with instant visual feedback on cart count and total.
- Update item quantities with increment/decrement controls and automatic price recalculation.
- Real-time price calculation including subtotal, shipping, and tax updates as items change.
- Free shipping threshold displayed and applied automatically when the order subtotal reaches \$75 or more.

4.4 Checkout System

- Multi-field shipping form with client-side validation using Zod schemas integrated with React Hook Form for robust, type-safe form handling.
- Order confirmation page displaying a unique order number in the format GJS-2026-XXXXXX.
- Estimated delivery date calculated and presented to the user upon order completion.
- Tax calculation at a flat 8% rate applied to the order subtotal, with a full price breakdown shown.

4.5 Order Management (My Account)

- Full order history accessible from the user's account page, listing all past orders chronologically.
- Expandable detail cards for each order, revealing items purchased, sizes, and quantities.
- Complete price breakdown per order — subtotal, shipping, tax, and total — visible in the expanded view.
- Shipping address stored and displayed per order for reference.
- Order status tracking with clear labels: Confirmed, Processing, Shipped, and Delivered.

4.6 Authentication

- Google Sign-In via Firebase OAuth 2.0 — one-click secure authentication with a trusted provider.
- Protected profile and orders page accessible only to authenticated users.
- Auto-fill of the checkout shipping form with the signed-in user's name and email, reducing friction.
- Session persistence so users remain logged in across page reloads and browser sessions.

4.7 Wishlist

- Save favorite products to a personal wishlist with a single click from product cards or pages.
- Wishlist items persist across sessions via localStorage, so saved jerseys remain on return visits.
- View all saved jerseys in a dedicated section within the user profile area.
- Easily move items from wishlist to cart when ready to purchase.

4.8 UI/UX Design

- Dark / light theme toggle allowing users to switch between a premium dark aesthetic and a clean light mode.
- Mobile-first responsive design ensuring the store looks and functions well on phones, tablets, and desktops.
- Premium dark aesthetic inspired by World Soccer Shop, with a sophisticated navy and red color palette.
- Shield-style logo (GJS) reinforcing the football / sports identity of the brand.
- Smooth animations and hover effects throughout for a polished, modern feel.
- Product badges including 'Trending', 'Out of Stock', and 'Sale' to communicate product status at a glance.

4.9 Admin Dashboard

- Overview dashboard providing a summary of orders and products in one centralized view.
- Accessible at the `/admin` route with role-based access control.
- Designed as a foundation for future expansion into full administrative CRUD operations.

5. Data Storage Explanation

A key architectural decision in this project is the intentional absence of a traditional database. This section explains what data the application manages, where each type of data is stored, and the rationale and limitations behind this approach.

5.1 What Data Exists and Where It Lives

Products & Teams	Hardcoded TS constants	Defined in <code>products.ts</code> and <code>teams.ts</code> — typed static data imported directly into components at build time.
Cart	Browser localStorage	Managed by a Zustand store with persist middleware; survives page reloads and browser restarts on the same device.
Wishlist	Browser localStorage	Persisted alongside cart via Zustand; saved jerseys remain across sessions.
Orders	Browser localStorage	Completed orders are stored locally with full detail (items, sizes, quantities, pricing, address, status) for the order history view.
User Identity	Firebase Authentication	Google OAuth user accounts are managed entirely by Firebase; no user profile data is stored in the app itself.

5.2 Why No Traditional Database?

No traditional database is used — this is intentional for demo simplicity. For a portfolio project, introducing a managed database (SQL or NoSQL) and a backend API would add significant infrastructure complexity (provisioning, migrations, connection management, hosting costs) without meaningfully improving the demonstration of front-end and integration skills. By using TypeScript constants for products and localStorage for user-specific state, the project remains fully functional, instantly deployable, and zero-cost to host — while still showcasing real e-commerce flows.

5.3 Known Limitations

- Data is device-specific: cart, wishlist, and orders stored in localStorage are not synced across devices or browsers. A user on their phone will not see orders placed on their laptop.
- No central inventory: because products are static, there is no real stock management or availability tracking tied to a backend.

- No order persistence beyond the browser: clearing browser data or switching devices erases order history.

5.4 Production Path

For a production application, the data layer would be replaced with a proper database such as PostgreSQL (e.g., via Supabase or PlanetScale) or MongoDB, accessed through a backend API. This would enable cross-device sync, real inventory management, persistent order records, and server-side business logic. The current architecture is designed so that swapping in a real backend would primarily involve replacing the Zustand persist calls with API calls — the component layer would remain largely unchanged.

6. Deployment & DevOps

The project uses a modern, automated deployment pipeline that ensures code quality and delivers updates to production with zero manual intervention. The workflow is built around GitHub as the source of truth, GitHub Actions for continuous integration, and Vercel for continuous deployment.

6.1 The Deployment Pipeline

GitHub Repository	The GitHub repository serves as the single source of truth for all code. All changes are committed and pushed to the main branch.
GitHub Actions CI	On every push, a GitHub Actions workflow automatically runs ESLint to catch linting issues and executes a full Next.js production build to verify the project compiles without errors.
Vercel Auto-Deploy	Once a push lands on the main branch, Vercel automatically triggers a new deployment, building and deploying the latest code to its global edge network.
Firebase Console	The Firebase Console manages authorized domains for OAuth, ensuring the production Vercel domain is permitted to perform Google sign-in.
Zero-Downtime Delivery	Vercel's edge network provides zero-downtime deployments — the new version goes live instantly with atomic switchover, and traffic is served from edge locations worldwide for low latency.

6.2 CI/CD Summary

- **Source control:** Git + GitHub — branch-based workflow with commits to main triggering the pipeline.
- **Continuous Integration:** GitHub Actions — ESLint + Next.js build on every push; failures block bad code from reaching production.
- **Continuous Deployment:** Vercel — automatic build and deploy on push to main, no manual steps required.
- **Authentication config:** Firebase Console — authorized domains managed for production OAuth.
- **Hosting:** Vercel edge network — global CDN distribution with automatic HTTPS and zero-downtime releases.

7. Challenges & Solutions

During development, several technical challenges arose that required investigation and problem-solving. Each challenge and its resolution is documented below.

7.1 @base-ui/react render prop vs. Radix UI asChild

Challenge	The @base-ui/react component library requires a 'render' prop for composition, whereas the more common Radix UI pattern uses an 'asChild' prop. This fundamental API difference affected 9+ component usages across the project that had been written following shadcn/ui (Radix-based) conventions.
Solution	Refactored all 9+ component usages to use the 'render' prop pattern instead of 'asChild'. Each component was reviewed and updated to pass the render function correctly, ensuring proper composition and styling passed through to child elements.

7.2 Firebase OAuth domain authorization for Vercel deployment

Challenge	After deploying to Vercel, Google OAuth sign-in failed because the production domain was not listed in Firebase's authorized domains. Firebase Authentication rejects OAuth requests originating from domains not in the authorized list, causing a redirect error.
Solution	Added the production Vercel domain to the authorized domains list in the Firebase Console (Authentication → Settings → Authorized domains). This allowed OAuth redirects to complete successfully from the live deployment.

7.3 Environment variable formatting issues

Challenge	Firebase configuration environment variables experienced formatting issues during deployment, causing the Firebase initialization to fail silently or use incorrect values, breaking authentication on the production build.
------------------	--

Solution

For demo reliability, the Firebase configuration was hardcoded directly in the source code rather than relying on environment variables. While not ideal for production (where secrets should remain in environment variables), this approach ensures the demo works consistently for all visitors and reviewers without environment-related failures.

8. Skills Demonstrated

This project demonstrates a broad range of modern web development skills spanning frontend architecture, state management, authentication, UI design, and DevOps. Each skill area below is showcased through concrete features in the application.

Modern React Development	React hooks (useState, useEffect, useMemo, useContext) with TypeScript for type-safe, reusable component logic.
Next.js App Router	File-based routing with layouts, nested routes, and server/client component separation using the latest Next.js 15 App Router paradigm.
State Management with Zustand	Lightweight, hook-based global state management with localStorage persistence middleware for cart, wishlist, and orders.
Form Validation with Zod & React Hook Form	Schema-based validation integrated with performant form state management for a robust, accessible checkout experience.
OAuth Authentication with Firebase	Secure Google OAuth 2.0 sign-in flow with session persistence and protected routes — a real-world authentication pattern.
Responsive UI Design with Tailwind CSS	Mobile-first, utility-first responsive design with a dark/light theme system and premium visual styling.
CI/CD Pipeline with GitHub Actions	Automated linting and build verification on every push, ensuring code quality and preventing broken deployments.
Deployment Workflow with Vercel	Zero-config continuous deployment with automatic builds, edge network distribution, and atomic zero-downtime releases.
Git Version Control & Collaborative Workflow	Structured commit history, branch-based development, and GitHub as a remote source of truth — industry-standard version control practices.

9. Future Improvements

While the current project is a fully functional portfolio demo, there are numerous directions in which it could be extended if developed further into a production application. The following improvements represent the most impactful next steps:

- **Real database** — Integrate PostgreSQL via Supabase or PlanetScale to replace localStorage with a persistent, cross-device data layer for products, orders, and user profiles.
- **Payment integration** — Add Stripe for real payment processing, enabling actual checkout transactions with credit card, Apple Pay, and Google Pay support.
- **Admin dashboard with full CRUD** — Expand the admin panel to support creating, updating, and deleting products and orders with a complete management interface.
- **Manager / Employee role dashboards** — Implement role-based dashboards with different permissions and views for managers versus employees.
- **Email notifications** — Send order confirmation, shipping, and delivery email notifications to customers using an email service provider.
- **Product reviews and ratings** — Allow authenticated users to leave star ratings and written reviews on purchased jerseys, building social proof.
- **Multi-language support** — Add internationalization (i18n) to serve users in multiple languages, appropriate for a global football audience.
- **Real inventory management** — Track stock levels per product and size, with automatic 'Out of Stock' badges and backorder handling tied to the database.

These improvements would transform the project from a portfolio demo into a production-ready e-commerce platform. The current architecture — particularly the clean separation between state management and component rendering — is designed to accommodate these upgrades incrementally without requiring a full rewrite.

10. Conclusion

Global Jersey Store is a comprehensive e-commerce web application that demonstrates a wide spectrum of modern web development skills in a single, cohesive project. From the initial product catalog and responsive UI design through to authentication, form-validated checkout, order management, and automated CI/CD deployment, the project covers the full lifecycle of a real-world web application.

10.1 What Was Built

- A full-featured e-commerce storefront with 50+ international football jerseys, filtering, search, and sorting capabilities.
- A complete shopping flow: product browsing → cart management → validated checkout → order confirmation with unique order numbers and delivery estimates.
- Google OAuth authentication via Firebase with protected account and order history pages.
- A persistent wishlist and cart system using Zustand with localStorage persistence.
- A polished, responsive UI with dark/light themes, premium styling, and smooth interactions.
- An automated deployment pipeline with GitHub Actions CI and Vercel CD.

10.2 What Was Learned

- How to architect a Next.js application using the App Router with file-based routing and layout composition.
- Practical state management with Zustand, including persisting state to localStorage and structuring stores for cart, wishlist, and orders.
- Integrating third-party authentication (Firebase OAuth) and handling domain authorization for production.
- Building accessible, validated forms with Zod schemas and React Hook Form.
- Adapting to library API differences (@base-ui/react's render prop vs. Radix's asChild) through systematic refactoring.
- Setting up a complete CI/CD pipeline from GitHub Actions through Vercel deployment.
- The trade-offs of a database-free architecture for demos versus production applications.

10.3 Closing Statement

As a portfolio piece, Global Jersey Store demonstrates not only technical proficiency across the modern web development stack but also the ability to make pragmatic architectural decisions, solve real integration challenges, and deliver a polished, deployable product. The project showcases end-to-end ownership — from design and development to testing, CI/CD, and deployment — making it a strong representation of full-stack web development capability. It stands as both a testament to the skills acquired and a foundation for continued growth as a developer.

Global Jersey Store

A portfolio project by Nossaiba El Asri — July 2026